Can you help me write a code to make a website that takes in a pdf (lecture notes) and another pdf (discussion questions / answers), and it creates another set of discussion questions around 2 to 3 questions that the original discussion sheet does not cover the contents that are in the pdf files. I would like to use the chatgpt tokens to create these new discussion sheets. Can you give me the code for the website that would creat it?

At first, I asked chat to write me a website that would get me 2 to 3 questions of the a new discussion sheet that would contain new concepts that has not been covered by discussion but covered by lecture.

Thought for 44s

Yep, this is totally doable. Let's build a simple *local* web app that:

1. Lets you upload
   - **Lecture notes PDF**
   - **Existing discussion sheet PDF (with Q/A)**
2. Extracts text from both PDFs.
3. Calls the **OpenAI ChatGPT API** (using your API key) to:
   - Find concepts in the lecture notes that *aren't* covered by existing questions.
   - Generate **2–3 new discussion questions + short answers**.
4. Shows the generated questions on the page.

I'll give you a minimal but complete Flask app you can run locally:

- `requirements.txt`
- `app.py` (Flask backend + OpenAI call + PDF parsing)
- `templates/index.html` (simple UI)

---

1. `requirements.txt`

Create a `requirements.txt` file:

txt Copy code

```
flask
openai>=1.0.0
PyPDF2
```

Install:

Copy code

Install:

bash Copy code

```
pip install -r requirements.txt
```

---

2. `app.py` (backend)

python Copy code

```python
import os
from flask import Flask, render_template, request
from openai import OpenAI
import PyPDF2

# ----------------------------
# Config
# ----------------------------
# Make sure you set your API key in the environment:
#   export OPENAI_API_KEY="sk-..."
client = OpenAI()  # Uses OPENAI_API_KEY from env

app = Flask(__name__)


def pdf_to_text(file_storage):
    """
    Read a PDF uploaded via <input type="file"> (Werkzeug FileStorage)
    and return its extracted text as a single string.
    """
    reader = PyPDF2.PdfReader(file_storage)
    text_chunks = []
    for page in reader.pages:
        page_text = page.extract_text() or ""
        text_chunks.append(page_text)
    return "\n".join(text_chunks)
```

Copy code

```
def pdf_to_text(file_storage):
    """
    Read a PDF uploaded via <input type="file"> (Werkzeug FileStorage)
    and return its extracted text as a single string.
    """
    reader = PyPDF2.PdfReader(file_storage)
    text_chunks = []
    for page in reader.pages:
        page_text = page.extract_text() or ""
        text_chunks.append(page_text)
    return "\n".join(text_chunks)


def generate_new_questions(lecture_text: str, sheet_text: str) -> str:
    """
    Call the OpenAI API to generate 2–3 new discussion questions
    that cover lecture content NOT already covered by the existing
    discussion sheet.
    """

    # Safety: avoid sending *huge* PDFs as-is
    max_chars = 15000
    lecture_text = lecture_text[:max_chars]
    sheet_text = sheet_text[:max_chars]

    prompt = f"""
You are helping a university TA create new discussion questions.

I will give you:
1. Lecture notes.
2. An existing discussion sheet that already contains questions and an

Your job:
- Identify important concepts in the lecture notes that are NOT alread
  the existing discussion questions.
- Then write 2–3 NEW discussion questions that focus on those uncovere

Constraints:
- Do NOT duplicate or lightly rephrase any existing discussion questio
- Aim for conceptual understanding (explain/interpret/compare/why), no
```

```
- Do NOT duplicate or lightly rephrase any existing discus
- Aim for conceptual understanding (explain/interpret/compare/why), no
  plug-and-chug calculations.
- Each question should be at the level of an upper-division math/CS/st
- Provide a short answer for each question.

Input:

=== LECTURE NOTES ===
\"\"\"{lecture_text}\"\"\"

=== EXISTING DISCUSSION SHEET (questions + answers) ===
\"\"\"{sheet_text}\"\"\"

Output format (exactly like this pattern):

Q1. <question text>
A1. <2–5 sentence answer>

Q2. <question text>
A2. <2–5 sentence answer>

Q3. <question text>
A3. <2–5 sentence answer>

Only output the questions and answers, nothing else.
"""

    completion = client.chat.completions.create(
        model="gpt-4o-mini",  # Cheap & solid for this task
        messages=[
            {
                "role": "developer",
                "content": (
                    "You write clear, rigorous, non-duplicative discus
                    "questions for university-level technical courses."
                ),
            },
            {
                "role": "user",
                "content": prompt,
```

Copy code

Copy code

```
                },
            ],
            max_completion_tokens=600,
            temperature=0.7,
        )

        return completion.choices[0].message.content


@app.route("/", methods=["GET", "POST"])
def index():
    generated = None
    error = None

    if request.method == "POST":
        lecture_pdf = request.files.get("lecture_pdf")
        discussion_pdf = request.files.get("discussion_pdf")

        if not lecture_pdf or not discussion_pdf:
            error = "Please upload both the lecture notes PDF and the d
        else:
            try:
                lecture_text = pdf_to_text(lecture_pdf)
                discussion_text = pdf_to_text(discussion_pdf)

                if not lecture_text.strip():
                    error = "Could not extract text from the lecture no
                elif not discussion_text.strip():
                    error = "Could not extract text from the discussion
                else:
                    generated = generate_new_questions(lecture_text, di

            except Exception as e:
                error = f"Something went wrong: {e}"

    return render_template("index.html", questions=generated, error=er


if __name__ == "__main__":
    # Run locally: python app.py
    app.run(debug=True)
```

Copy code

3. `templates/index.html` (frontend)

Create a folder `templates/` next to `app.py`, and inside it put `index.html`:

html

Copy code

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <title>Discussion Question Generator</title>
  <style>
    body {
      font-family: system-ui, -apple-system, BlinkMacSystemFont, "Sego
      max-width: 800px;
      margin: 40px auto;
      padding: 0 16px;
      line-height: 1.5;
    }
    h1 {
      margin-bottom: 0.5rem;
    }
    .card {
      border: 1px solid #ddd;
      border-radius: 8px;
      padding: 16px;
      margin-top: 20px;
    }
    label {
      display: block;
      margin-top: 12px;
      font-weight: 500;
    }
    input[type="file"] {
      margin-top: 4px;
    }
    button {
      margin-top: 16px;
```

Copy code

```
      margin-top: 16px;
      padding: 8px 16px;
      border-radius: 6px;
      border: none;
      cursor: pointer;
      font-size: 14px;
      font-weight: 600;
      background: #2563eb;
      color: white;
    }
    button:hover {
      background: #1d4ed8;
    }
    .error {
      color: #b91c1c;
      margin-top: 12px;
      font-weight: 500;
    }
    pre {
      white-space: pre-wrap;
      background: #f9fafb;
      padding: 12px;
      border-radius: 6px;
      border: 1px solid #e5e7eb;
    }
  </style>
</head>
<body>
  <h1>Discussion Question Generator</h1>
  <p>
    Upload your <strong>lecture notes PDF</strong> and your
    <strong>existing discussion sheet PDF</strong>. The tool will
    generate 2–3 new discussion questions (with short answers) that
    target content not already covered in the original sheet.
  </p>

  <div class="card">
    <form method="post" enctype="multipart/form-data">
      <label>
        Lecture notes PDF:
        <input type="file" name="lecture_pdf" accept="application/pdf"
```

```
        <input type="file" name="lecture_pdf" accept="application/pdf"
      </label>

      <label>
        Discussion questions/answers PDF:
        <input type="file" name="discussion_pdf" accept="application/p
      </label>

      <button type="submit">Generate new questions</button>
    </form>

    {% if error %}
      <div class="error">{{ error }}</div>
    {% endif %}
  </div>

  {% if questions %}
    <div class="card">
      <h2>Generated Discussion Questions</h2>
      <pre>{{ questions }}</pre>
    </div>
  {% endif %}
</body>
</html>
```

Copy code

---

4. How to run it

From the directory with `app.py`:

bash

```
export OPENAI_API_KEY="sk-..."   # your key
python app.py
```

Copy code

Then open:

http://127.0.0.1:5000/ ↗ in your browser.

---

5. Tweaks you can make

4. How to run it

From the directory with `app.py`:

bash Copy code

```
export OPENAI_API_KEY="sk-..."   # your key
python app.py
```

Then open:
http://127.0.0.1:5000/ ↗ in your browser.

---

5. Tweaks you can make

- **Change model / cost**:
  - For cheaper runs, stick with `gpt-4o-mini`. OpenAI Platform
  - For higher quality, you can swap to a larger model like `gpt-4o`.
- **Adjust truncation** (`max_chars`):
  If your notes are long and you're hitting context limits, tune `max_chars` or add a summary step before question generation.
- **Change output style**:
  Adjust the prompt to:
  - Only output questions (no answers).
  - Output in Markdown list format.
  - Target a specific course (e.g., "CS 182 deep learning").

If you tell me your preferred stack (React/Next.js + a Python/Node backend, etc.), I can also give you a more production-style version with separate frontend + API route.

Sources

how to use open api key

Here's the simplest, clean, *step-by-step* explanation of **how to use an OpenAI API key** — both for *local development* and *website/backend code*.

✅ 1. Get your API key from OpenAI

1. Go to: https://platform.openai.com/settings/organization/api-keys ↗
2. Click **Create new secret key**
3. Copy the key — it looks like:

Copy code

```
sk-proj-xxxxxxxxxxxxxxxxxxxx
```

⚠️ **Never hardcode this key in code that will be pushed to GitHub.**

⚠️ Treat it like a password.

✅ 2. Store the key securely on your computer

Mac / Linux

Open a terminal and run:

bash Copy code

```bash
export OPENAI_API_KEY="sk-...."
```

To make it permanent, add it to your shell config:

bash Copy code

```bash
echo 'export OPENAI_API_KEY="sk-...."' >> ~/.zshrc
source ~/.zshrc
```

Windows PowerShell

powershell Copy code

```powershell
setx OPENAI_API_KEY "sk-...."
```

✅ 3. Use the key in Python

Copy code

Windows PowerShell

powershell Copy code

```
setx OPENAI_API_KEY "sk-...."
```

✅ 3. Use the key in Python

OpenAI's new SDK automatically loads the key from environment variables.

Example:

python Copy code

```
from openai import OpenAI

client = OpenAI()  # automatically uses OPENAI_API_KEY env variable

completion = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role":"user","content":"Hello!"}]
)

print(completion.choices[0].message.content)
```

No need to pass the key manually.

✅ 4. Use the key in JavaScript (Node.js)

Install:

bash Copy code

```
npm install openai
```

Then:

javascript Copy code

javascript Copy code

```javascript
import OpenAI from "openai";

const client = new OpenAI({
  apiKey: process.env.OPENAI_API_KEY,
});

const response = await client.chat.completions.create({
  model: "gpt-4o-mini",
  messages: [{ role: "user", content: "Hello!" }],
});

console.log(response.choices[0].message.content);
```

Run with:

bash Copy code

```bash
OPENAI_API_KEY="sk-..." node app.js
```

✅ 5. Use the key in a Flask web app (your use case)

In your `app.py` :

python Copy code

```python
from openai import OpenAI
client = OpenAI()  # uses your API key from environment
```

Launch with:

bash Copy code

```bash
export OPENAI_API_KEY="sk-..."
python app.py
```

❌ DO NOT DO THIS (

❌ DO NOT DO THIS (unsafe)

python Copy code

```python
client = OpenAI(api_key="sk-...")  # avoid hardcoding
```

javascript Copy code

```javascript
const client = new OpenAI({ apiKey: "sk-..." }); // unsafe
```

Hardcoding = dangerous, especially if stored in GitHub.

⭐ QUICK TEST (verify your key)

Try:

bash Copy code

```bash
curl https://api.openai.com/v1/models \
  -H "Authorization: Bearer $OPENAI_API_KEY"
```

If your key works, you'll get a JSON list of models.

If you want, I can also:

- ✅ help you set up `.env` files
- ✅ create a secure backend API wrapper
- ✅ integrate your website with the OpenAI API
- ✅ add rate limits & logging
- ✅ deploy it to Render/Vercel/AWS

Just tell me what stack you want your website to run on (Flask? Node? Next.js? React + Python?).

The lecture notes are notes that are written by hand from app like goodnotes or notability, so it will be hard to extract text if we are trying to get it like that. Can you modify the code such that it can handle this new situation?

The current code was not able to read the notes because it was hand-written, so i prompted gpt to have it adjust such that it can read hand-written notes.

Thought for 15s

Nice, that's a super common constraint with GoodNotes/Notability PDFs.

Since those lecture-note PDFs are basically images of handwriting, `PyPDF2` won't see any text. We need to:

- Keep using **normal text extraction** for the discussion sheet (since it's probably typed).
- Use **OCR** for the handwritten lecture notes.

The easiest way: add `pdf2image + pytesseract` to your stack so we can convert the PDF pages to images and run OCR over them.

Below is a full updated version of the code.

---

1. Updated `requirements.txt`

txt Copy code

```
flask
openai>=1.0.0
PyPDF2
pdf2image
pytesseract
```

Install:

bash Copy code

```
pip install -r requirements.txt
```

Install:

bash Copy code

```
pip install -r requirements.txt
```

You'll also need system packages:

macOS (Homebrew)

bash Copy code

```
brew install poppler tesseract
```

Ubuntu / Debian-ish

bash Copy code

```
sudo apt-get update
sudo apt-get install poppler-utils tesseract-ocr
```

(You only do this once per machine.)

---

2. Updated `app.py` (with OCR for lecture notes)

This version:

- Uses **OCR for the lecture notes PDF**.
- Uses regular `PyPDF2` text extraction for the discussion sheet PDF.
- Everything else (Flask routes, OpenAI usage) is the same idea as before.

python Copy code

```
import os
from flask import Flask, render_template, request
from openai import OpenAI
import PyPDF2
from pdf2image import convert_from_bytes
import pytesseract

# ----------------------------
```

Copy code

```
from pdf2image import convert_from_bytes
import pytesseract

# ----------------------------
# Config
# ----------------------------
# Make sure you set your API key in the environment:
#   export OPENAI_API_KEY="sk-..."
client = OpenAI()  # Uses OPENAI_API_KEY from env

app = Flask(__name__)


def pdf_to_text_standard(file_storage):
    """
    Extract text from a 'normal' PDF (with embedded text),
    e.g., a typed discussion sheet.
    """
    file_storage.stream.seek(0)  # ensure start of file
    reader = PyPDF2.PdfReader(file_storage.stream)
    text_chunks = []
    for page in reader.pages:
        page_text = page.extract_text() or ""
        text_chunks.append(page_text)
    return "\n".join(text_chunks)


def pdf_to_text_ocr(file_storage):
    """
    Extract text via OCR from a 'scanned' or handwritten PDF,
    e.g., GoodNotes / Notability lecture notes.
    - Converts pages to images using pdf2image.
    - Runs pytesseract on each page.
    """
    # Read the bytes from the uploaded file
    file_storage.stream.seek(0)
    pdf_bytes = file_storage.read()

    # Convert each page to a PIL Image
    images = convert_from_bytes(pdf_bytes)
```

Copy code

```
    """
    Extract text via OCR from a 'scanned' or handwritten PDF,
    e.g., GoodNotes / Notability lecture notes.
    - Converts pages to images using pdf2image.
    - Runs pytesseract on each page.
    """
    # Read the bytes from the uploaded file
    file_storage.stream.seek(0)
    pdf_bytes = file_storage.read()

    # Convert each page to a PIL Image
    images = convert_from_bytes(pdf_bytes)

    text_chunks = []
    for img in images:
        # You can tweak languages / config here if needed
        page_text = pytesseract.image_to_string(img)
        text_chunks.append(page_text)

    return "\n".join(text_chunks)


def generate_new_questions(lecture_text: str, sheet_text: str) -> str:
    """
    Call the OpenAI API to generate 2–3 new discussion questions
    that cover lecture content NOT already covered by the existing
    discussion sheet.
    """

    # Safety: avoid sending *huge* PDFs as-is
    max_chars = 15000
    lecture_text = lecture_text[:max_chars]
    sheet_text = sheet_text[:max_chars]

    prompt = f"""
You are helping a university TA create new discussion questions.

I will give you:
1. Lecture notes (transcribed from handwritten notes, so there may be (
2. An existing discussion sheet that already contains questions and ans
```

Copy code

```
I will give you:
1. Lecture notes (transcribed from handwritten notes, so there may be (
2. An existing discussion sheet that already contains questions and ans

Your job:
- Identify important concepts in the lecture notes that are NOT already
  the existing discussion questions.
- Then write 2–3 NEW discussion questions that focus on those uncovered

Constraints:
- Do NOT duplicate or lightly rephrase any existing discussion questio
- Aim for conceptual understanding (explain/interpret/compare/why), no
  plug-and-chug calculations.
- Each question should be at the level of an upper-division math/CS/sta
- Provide a short answer for each question.
- If the OCR text is messy, do your best to reconstruct the intended ma

Input:

=== LECTURE NOTES (OCR TEXT) ===
\"\"\"{lecture_text}\"\"\"

=== EXISTING DISCUSSION SHEET (questions + answers) ===
\"\"\"{sheet_text}\"\"\"

Output format (exactly like this pattern):

Q1. <question text>
A1. <2–5 sentence answer>

Q2. <question text>
A2. <2–5 sentence answer>

Q3. <question text>
A3. <2–5 sentence answer>

Only output the questions and answers, nothing else.
"""

    completion = client.chat.completions.create(
        model="gpt-4o-mini",
```

Copy code

```
A2. <2–5 sentence answer>

Q3. <question text>
A3. <2–5 sentence answer>

Only output the questions and answers, nothing else.
"""

    completion = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {
                "role": "developer",
                "content": (
                    "You write clear, rigorous, non-duplicative discus
                    "questions for university-level technical courses."
                ),
            },
            {
                "role": "user",
                "content": prompt,
            },
        ],
        max_completion_tokens=600,
        temperature=0.7,
    )

    return completion.choices[0].message.content


@app.route("/", methods=["GET", "POST"])
def index():
    generated = None
    error = None

    if request.method == "POST":
        lecture_pdf = request.files.get("lecture_pdf")
        discussion_pdf = request.files.get("discussion_pdf")

        if not lecture_pdf or not discussion_pdf:
            error = "Please upload both the lecture notes PDF and the d
```

Copy code

```
@app.route("/", methods=["GET", "POST"])
def index():
    generated = None
    error = None

    if request.method == "POST":
        lecture_pdf = request.files.get("lecture_pdf")
        discussion_pdf = request.files.get("discussion_pdf")

        if not lecture_pdf or not discussion_pdf:
            error = "Please upload both the lecture notes PDF and the d
        else:
            try:
                # Handwritten lecture notes → OCR
                lecture_text = pdf_to_text_ocr(lecture_pdf)

                # Typed discussion sheet → normal extraction
                discussion_text = pdf_to_text_standard(discussion_pdf)

                if not lecture_text.strip():
                    error = "Could not extract text from the lecture no
                elif not discussion_text.strip():
                    error = "Could not extract text from the discussion
                else:
                    generated = generate_new_questions(lecture_text, di

            except Exception as e:
                error = f"Something went wrong: {e}"

    return render_template("index.html", questions=generated, error=er


if __name__ == "__main__":
    app.run(debug=True)
```

---

3. `templates/index.html` (unchanged except maybe wording)

You can keep the previous HTML; if you want to hint that lecture notes can be

Copy code

```
if __name__ == "__main__":
    app.run(debug=True)
```

---

3. `templates/index.html` (unchanged except maybe wording)

You can keep the previous HTML; if you want to hint that lecture notes can be handwritten, tweak the text slightly:

html Copy code

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <title>Discussion Question Generator</title>
  <style>
    body {
      font-family: system-ui, -apple-system, BlinkMacSystemFont, "Sego
      max-width: 800px;
      margin: 40px auto;
      padding: 0 16px;
      line-height: 1.5;
    }
    h1 {
      margin-bottom: 0.5rem;
    }
    .card {
      border: 1px solid #ddd;
      border-radius: 8px;
      padding: 16px;
      margin-top: 20px;
    }
    label {
      display: block;
      margin-top: 12px;
      font-weight: 500;
    }
    input[type="file"] {
```

Copy code

```
      border: 1px solid #ddd;
      border-radius: 8px;
      padding: 16px;
      margin-top: 20px;
    }
    label {
      display: block;
      margin-top: 12px;
      font-weight: 500;
    }
    input[type="file"] {
      margin-top: 4px;
    }
    button {
      margin-top: 16px;
      padding: 8px 16px;
      border-radius: 6px;
      border: none;
      cursor: pointer;
      font-size: 14px;
      font-weight: 600;
      background: #2563eb;
      color: white;
    }
    button:hover {
      background: #1d4ed8;
    }
    .error {
      color: #b91c1c;
      margin-top: 12px;
      font-weight: 500;
    }
    pre {
      white-space: pre-wrap;
      background: #f9fafb;
      padding: 12px;
      border-radius: 6px;
      border: 1px solid #e5e7eb;
    }
  </style>
</head>
```

Copy code

```
      margin-top: 12px;
      font-weight: 500;
    }
    pre {
      white-space: pre-wrap;
      background: #f9fafb;
      padding: 12px;
      border-radius: 6px;
      border: 1px solid #e5e7eb;
    }
  </style>
</head>
<body>
  <h1>Discussion Question Generator</h1>
  <p>
    Upload your <strong>lecture notes PDF</strong> (handwritten is fin
    and your <strong>existing discussion sheet PDF</strong>. The tool
    discussion questions (with short answers) that target content not
    the original sheet.
  </p>

  <div class="card">
    <form method="post" enctype="multipart/form-data">
      <label>
        Lecture notes PDF (handwritten OK):
        <input type="file" name="lecture_pdf" accept="application/pdf"
      </label>

      <label>
        Discussion questions/answers PDF:
        <input type="file" name="discussion_pdf" accept="application/p
      </label>

      <button type="submit">Generate new questions</button>
    </form>

    {% if error %}
      <div class="error">{{ error }}</div>
    {% endif %}
  </div>
```

```
      </label>

      <button type="submit">Generate new questions</button>
    </form>

    {% if error %}
      <div class="error">{{ error }}</div>
    {% endif %}
  </div>

  {% if questions %}
    <div class="card">
      <h2>Generated Discussion Questions</h2>
      <pre>{{ questions }}</pre>
    </div>
  {% endif %}
</body>
</html>
```

Copy code

---

4. Run everything

From the project folder:

bash

```
export OPENAI_API_KEY="sk-..."   # or set it in your shell config
python app.py
```

Copy code

Then open:
http://127.0.0.1:5000/↗

Upload:

- Lecture notes exported from GoodNotes/Notability as PDF.
- Your discussion sheet PDF.

It'll OCR the notes, extract the discussion questions, and ask ChatGPT to synthesize **2–3 new questions + answers** that hit uncovered material.

Then open:

http://127.0.0.1:5000/↗

Upload:

- Lecture notes exported from GoodNotes/Notability as PDF.
- Your discussion sheet PDF.

It'll OCR the notes, extract the discussion questions, and ask ChatGPT to synthesize **2–3 new questions + answers** that hit uncovered material.

If you want to go further, we can:

- Add a toggle: "Lecture notes are handwritten vs typed" (to switch OCR on/off).
- Cache OCR so you don't pay that cost every time for the same lecture.
- Export the generated questions as a nicely formatted PDF/LaTeX handout.

Can you make questions and answer for the new discussion sheet and make them into a pdf such that I can download and use it offline?

It created discussion, but it created it such that it was just questions on the website, but I wanted it to be pdf such that it's easier for me to carry around.

Thought for 7s ›

Yes! We can extend the app so that:

1. It generates **new questions + answers** using ChatGPT (like before), and
2. It lets you **download them as a nicely formatted PDF** to use offline.

We'll do this by:

- Keeping the existing "upload PDFs → generate questions" route.
- Adding a **second route** `/download_pdf` that:
  - Takes the generated Q&A text.
  - Uses `reportlab` to turn it into a PDF.
  - Returns it as a file download.

Below is a full working setup.

Yes! We can extend the app so that:

1. It generates **new questions + answers** using ChatGPT (like before), and
2. It lets you **download them as a nicely formatted PDF** to use offline.

We'll do this by:

- Keeping the existing "upload PDFs → generate questions" route.
- Adding a **second route** `/download_pdf` that:
  - Takes the generated Q&A text.
  - Uses `reportlab` to turn it into a PDF.
  - Returns it as a file download.

Below is a full working setup.

---

1. Updated `requirements.txt`

Add `reportlab` :

txt Copy code

```
flask
openai>=1.0.0
PyPDF2
pdf2image
pytesseract
reportlab
```

Install:

bash Copy code

```
pip install -r requirements.txt
```

And (once per machine) install the system tools for OCR (if not already done):

macOS (Homebrew):

bash Copy code

```
brew install poppler tesseract
```

Copy code

```
pip install -r requirements.txt
```

And (once per machine) install the system tools for OCR (if not already done):

macOS (Homebrew):

bash Copy code

```
brew install poppler tesseract
```

Ubuntu / Debian:

bash Copy code

```
sudo apt-get update
sudo apt-get install poppler-utils tesseract-ocr
```

---

2. Updated `app.py` (now with "download PDF" route)

python Copy code

```python
import os
from io import BytesIO

from flask import Flask, render_template, request, send_file
from openai import OpenAI
import PyPDF2
from pdf2image import convert_from_bytes
import pytesseract

from reportlab.lib.pagesizes import letter
from reportlab.pdfgen import canvas

# ----------------------------
# Config
# ----------------------------
# Make sure you set your API key in the environment:
#   export OPENAI_API_KEY="sk-..."
client = OpenAI()  # Uses OPENAI_API_KEY from env
```

Copy code

```
import PyPDF2
from pdf2image import convert_from_bytes
import pytesseract

from reportlab.lib.pagesizes import letter
from reportlab.pdfgen import canvas

# ----------------------------
# Config
# ----------------------------
# Make sure you set your API key in the environment:
#   export OPENAI_API_KEY="sk-..."
client = OpenAI()  # Uses OPENAI_API_KEY from env

app = Flask(__name__)


# ----------------------------
# PDF extraction helpers
# ----------------------------

def pdf_to_text_standard(file_storage):
    """
    Extract text from a 'normal' PDF (with embedded text),
    e.g., a typed discussion sheet.
    """
    file_storage.stream.seek(0)  # ensure start of file
    reader = PyPDF2.PdfReader(file_storage.stream)
    text_chunks = []
    for page in reader.pages:
        page_text = page.extract_text() or ""
        text_chunks.append(page_text)
    return "\n".join(text_chunks)


def pdf_to_text_ocr(file_storage):
    """
    Extract text via OCR from a 'scanned' or handwritten PDF,
    e.g., GoodNotes / Notability lecture notes.
    - Converts pages to images using pdf2image.
    - Runs pytesseract on each page.
```

Copy code

```
    for page in reader.pages:
        page_text = page.extract_text() or ""
        text_chunks.append(page_text)
    return "\n".join(text_chunks)


def pdf_to_text_ocr(file_storage):
    """
    Extract text via OCR from a 'scanned' or handwritten PDF,
    e.g., GoodNotes / Notability lecture notes.
    - Converts pages to images using pdf2image.
    - Runs pytesseract on each page.
    """
    file_storage.stream.seek(0)
    pdf_bytes = file_storage.read()

    images = convert_from_bytes(pdf_bytes)
    text_chunks = []
    for img in images:
        page_text = pytesseract.image_to_string(img)
        text_chunks.append(page_text)

    return "\n".join(text_chunks)


# ----------------------------
# OpenAI helper
# ----------------------------

def generate_new_questions(lecture_text: str, sheet_text: str) -> str:
    """
    Call the OpenAI API to generate 2–3 new discussion questions
    that cover lecture content NOT already covered by the existing
    discussion sheet.
    """

    # Safety: avoid sending *huge* PDFs as-is
    max_chars = 15000
    lecture_text = lecture_text[:max_chars]
    sheet_text = sheet_text[:max_chars]
```

Copy code

```
    """
    Call the OpenAI API to generate 2–3 new discussion questions
    that cover lecture content NOT already covered by the existing
    discussion sheet.
    """

    # Safety: avoid sending *huge* PDFs as-is
    max_chars = 15000
    lecture_text = lecture_text[:max_chars]
    sheet_text = sheet_text[:max_chars]

    prompt = f"""
You are helping a university TA create new discussion questions.

I will give you:
1. Lecture notes (transcribed from handwritten notes, so there may be (
2. An existing discussion sheet that already contains questions and ans

Your job:
- Identify important concepts in the lecture notes that are NOT already
  the existing discussion questions.
- Then write 2–3 NEW discussion questions that focus on those uncovered

Constraints:
- Do NOT duplicate or lightly rephrase any existing discussion questio
- Aim for conceptual understanding (explain/interpret/compare/why), no
  plug-and-chug calculations.
- Each question should be at the level of an upper-division math/CS/sta
- Provide a short answer for each question.
- If the OCR text is messy, do your best to reconstruct the intended ma

Input:

=== LECTURE NOTES (OCR TEXT) ===
\"\"\"{lecture_text}\"\"\"

=== EXISTING DISCUSSION SHEET (questions + answers) ===
\"\"\"{sheet_text}\"\"\"

Output format (exactly like this pattern):
```

Copy code

```
- If the OCR text is messy, do your best to reconstruct the intended ma

Input:

=== LECTURE NOTES (OCR TEXT) ===
\"\"\"{lecture_text}\"\"\"

=== EXISTING DISCUSSION SHEET (questions + answers) ===
\"\"\"{sheet_text}\"\"\"

Output format (exactly like this pattern):

Q1. <question text>
A1. <2–5 sentence answer>

Q2. <question text>
A2. <2–5 sentence answer>

Q3. <question text>
A3. <2–5 sentence answer>

Only output the questions and answers, nothing else.
"""

    completion = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {
                "role": "developer",
                "content": (
                    "You write clear, rigorous, non-duplicative discus
                    "questions for university-level technical courses."
                ),
            },
            {
                "role": "user",
                "content": prompt,
            },
        ],
        max_completion_tokens=600,
        temperature=0.7,
```

Copy code

```
                    "role": "developer",
                    "content": (
                        "You write clear, rigorous, non-duplicative discus
                        "questions for university-level technical courses."
                    ),
                },
                {
                    "role": "user",
                    "content": prompt,
                },
            ],
            max_completion_tokens=600,
            temperature=0.7,
        )

        return completion.choices[0].message.content


# ----------------------------
# Flask routes
# ----------------------------

@app.route("/", methods=["GET", "POST"])
def index():
    generated = None
    error = None

    if request.method == "POST":
        lecture_pdf = request.files.get("lecture_pdf")
        discussion_pdf = request.files.get("discussion_pdf")

        if not lecture_pdf or not discussion_pdf:
            error = "Please upload both the lecture notes PDF and the d
        else:
            try:
                # Handwritten lecture notes → OCR
                lecture_text = pdf_to_text_ocr(lecture_pdf)

                # Typed discussion sheet → normal extraction
                discussion_text = pdf_to_text_standard(discussion_pdf)
```

Copy code

```
        if not lecture_pdf or not discussion_pdf:
            error = "Please upload both the lecture notes PDF and the d
        else:
            try:
                # Handwritten lecture notes → OCR
                lecture_text = pdf_to_text_ocr(lecture_pdf)

                # Typed discussion sheet → normal extraction
                discussion_text = pdf_to_text_standard(discussion_pdf)

                if not lecture_text.strip():
                    error = "Could not extract text from the lecture no
                elif not discussion_text.strip():
                    error = "Could not extract text from the discussion
                else:
                    generated = generate_new_questions(lecture_text, di

            except Exception as e:
                error = f"Something went wrong: {e}"

    # Render page with optional questions + error
    return render_template("index.html", questions=generated, error=er


@app.route("/download_pdf", methods=["POST"])
def download_pdf():
    """
    Take the generated questions/answers text (posted from the form),
    render it into a PDF, and send it as a downloadable file.
    """
    questions_text = request.form.get("questions_text", "")

    if not questions_text.strip():
        # Fallback: if somehow empty, just redirect back or return simp
        # but for simplicity we'll just return a tiny PDF with a messag
        questions_text = "No questions were provided."

    # Create PDF in memory
    buffer = BytesIO()
    c = canvas.Canvas(buffer, pagesize=letter)
```

Copy code

```
    """
    Take the generated questions/answers text (posted from the form),
    render it into a PDF, and send it as a downloadable file.
    """
    questions_text = request.form.get("questions_text", "")

    if not questions_text.strip():
        # Fallback: if somehow empty, just redirect back or return simp
        # but for simplicity we'll just return a tiny PDF with a messag
        questions_text = "No questions were provided."

    # Create PDF in memory
    buffer = BytesIO()
    c = canvas.Canvas(buffer, pagesize=letter)
    width, height = letter

    # Start drawing text
    textobject = c.beginText()
    textobject.setTextOrigin(72, height - 72)  # 1-inch margins
    textobject.setLeading(14)  # line spacing

    # Simple line-by-line write, with manual page breaks
    for line in questions_text.split("\n"):
        # If we're near the bottom, start a new page
        if textobject.getY() <= 72:
            c.drawText(textobject)
            c.showPage()
            textobject = c.beginText()
            textobject.setTextOrigin(72, height - 72)
            textobject.setLeading(14)
        textobject.textLine(line)

    c.drawText(textobject)
    c.save()

    buffer.seek(0)

    return send_file(
        buffer,
        as_attachment=True,
        download_name="new_discussion_sheet.pdf",
```

```
            c.showPage()
            textobject = c.beginText()
            textobject.setTextOrigin(72, height - 72)
            textobject.setLeading(14)
        textobject.textLine(line)

    c.drawText(textobject)
    c.save()

    buffer.seek(0)

    return send_file(
        buffer,
        as_attachment=True,
        download_name="new_discussion_sheet.pdf",
        mimetype="application/pdf",
    )


if __name__ == "__main__":
    app.run(debug=True)
```

Copy code

---

3. Updated `templates/index.html`

Key changes:

- Still shows the generated Q&A on the page.
- Adds a **"Download as PDF"** button that posts the Q&A text to `/download_pdf` .

Create or update `templates/index.html` :

html

Copy code

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <title>Discussion Question Generator</title>
  <style>
    body {
      font-family: system-ui, -apple-system, BlinkMacSystemFont, "Sego
```

- Still shows the generated Q&A on the page.
- Adds a **"Download as PDF"** button that posts the Q&A text to `/download_pdf`.

Create or update `templates/index.html`:

html Copy code

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <title>Discussion Question Generator</title>
  <style>
    body {
      font-family: system-ui, -apple-system, BlinkMacSystemFont, "Sego
      max-width: 800px;
      margin: 40px auto;
      padding: 0 16px;
      line-height: 1.5;
    }
    h1 {
      margin-bottom: 0.5rem;
    }
    .card {
      border: 1px solid #ddd;
      border-radius: 8px;
      padding: 16px;
      margin-top: 20px;
    }
    label {
      display: block;
      margin-top: 12px;
      font-weight: 500;
    }
    input[type="file"] {
      margin-top: 4px;
    }
    button {
      margin-top: 16px;
      padding: 8px 16px;
      border-radius: 6px;
      border: none;
```

Copy code

```
      display: block;
      margin-top: 12px;
      font-weight: 500;
    }
    input[type="file"] {
      margin-top: 4px;
    }
    button {
      margin-top: 16px;
      padding: 8px 16px;
      border-radius: 6px;
      border: none;
      cursor: pointer;
      font-size: 14px;
      font-weight: 600;
      background: #2563eb;
      color: white;
    }
    button:hover {
      background: #1d4ed8;
    }
    .secondary-button {
      background: #6b7280;
      margin-left: 8px;
    }
    .secondary-button:hover {
      background: #4b5563;
    }
    .error {
      color: #b91c1c;
      margin-top: 12px;
      font-weight: 500;
    }
    pre {
      white-space: pre-wrap;
      background: #f9fafb;
      padding: 12px;
      border-radius: 6px;
      border: 1px solid #e5e7eb;
    }
  </style>
```

Copy code

```
      color: #b91c1c;
      margin-top: 12px;
      font-weight: 500;
    }
    pre {
      white-space: pre-wrap;
      background: #f9fafb;
      padding: 12px;
      border-radius: 6px;
      border: 1px solid #e5e7eb;
    }
  </style>
</head>
<body>
  <h1>Discussion Question Generator</h1>
  <p>
    Upload your <strong>lecture notes PDF</strong> (handwritten is fin
    and your <strong>existing discussion sheet PDF</strong>. The tool
    discussion questions (with short answers) that target content not
    the original sheet. You can then download the result as a PDF to u
  </p>

  <div class="card">
    <form method="post" enctype="multipart/form-data">
      <label>
        Lecture notes PDF (handwritten OK):
        <input type="file" name="lecture_pdf" accept="application/pdf"
      </label>

      <label>
        Discussion questions/answers PDF:
        <input type="file" name="discussion_pdf" accept="application/p
      </label>

      <button type="submit">Generate new questions</button>
    </form>

    {% if error %}
      <div class="error">{{ error }}</div>
    {% endif %}
  </div>
```

Copy code

```
      <label>
        Discussion questions/answers PDF:
        <input type="file" name="discussion_pdf" accept="application/p
      </label>

      <button type="submit">Generate new questions</button>
    </form>

    {% if error %}
      <div class="error">{{ error }}</div>
    {% endif %}
  </div>

  {% if questions %}
    <div class="card">
      <h2>Generated Discussion Questions</h2>
      <pre>{{ questions }}</pre>

      <!-- Form to download as PDF -->
      <form method="post" action="/download_pdf">
        <!-- Hidden field with the generated questions text -->
        <textarea name="questions_text" style="display:none;">{{ quest
        <button type="submit" class="secondary-button">Download as PDF
      </form>
    </div>
  {% endif %}
</body>
</html>
```

---

4. How you actually use it

1. Start the server:

bash Copy code

```
export OPENAI_API_KEY="sk-..."
python app.py
```

2. Go to:

```
</body>
</html>
```

Copy code

---

4. How you actually use it

1. Start the server:

bash Copy code

```
export OPENAI_API_KEY="sk-..."
python app.py
```

2. Go to:
   http://127.0.0.1:5000/↗
3. Upload:
   - Lecture notes PDF (GoodNotes / Notability export).
   - Existing discussion Q&A PDF.
4. Click **"Generate new questions"**.
5. Scroll down:
   - You'll see the generated Q&A in text form.
   - Click **"Download as PDF"** → `new_discussion_sheet.pdf` downloads.
6. Print it, share it, or use offline.

If you want a fancier layout (logo, course header, page numbers, etc.), we can tweak the `download_pdf` function to format the PDF more like a real worksheet (titles, spacing, question numbering, etc.).

it seems like the questions are more conceptual questions for the new discusison sheets. Would it be possible to have a mixture of conceptual and image + questions just like questions in the old discussions?

Thought for 37s ›

question numbering, etc.).

it seems like the questions are more conceptual questions for the new discusison sheets. Would it be possible to have a mixture of conceptual and image + questions just like questions in the old discussions?

I noticed that the questions were simple conceptual questions that were not covered by discussion, but i wanted them to be more specific just like the discussions that we got.

Thought for 37s

Yes, totally possible — we can make the app generate **a mix of conceptual questions and "image + question" problems**, similar to what's on your original discussion sheets.

Below I'll show you how to:

1. Change the **question generator** so it produces both:
   - plain conceptual Qs, and
   - image-based Qs (with an image description).
2. Use the **OpenAI Images API** to actually generate images for the image-based questions.
3. Display those images + questions on the website.
   (We'll keep the PDF export as text-only for now; you can still print it and manually attach the original/discussion images, or we can later extend it to embed images into the PDF as well.)

---

1. Update `requirements.txt`

Make sure you have these (building on what we had before):

txt Copy code

```
flask
openai>=1.0.0
PyPDF2
pdf2image
```

into the PDF as well.)

---

1. Update `requirements.txt`

Make sure you have these (building on what we had before):

txt Copy code

```
flask
openai>=1.0.0
PyPDF2
pdf2image
pytesseract
reportlab
```

You already know the system deps for OCR:

- macOS: `brew install poppler tesseract`
- Ubuntu: `sudo apt-get install poppler-utils tesseract-ocr`

---

2. Updated `app.py` (conceptual + image questions)

Here's an updated `app.py` that:

- Uses OCR for handwritten lecture notes.
- Uses normal PDF text extraction for the discussion sheet.
- Asks ChatGPT to output **structured JSON** with both conceptual and image questions.
- For questions of type `"image"`, uses `gpt-image-1` to generate an image and saves it under `static/generated/`. OpenAI Platform
- Displays everything on the page.
- Still supports "Download as PDF" (text-only Q&A).

python Copy code

```
import os
import json
import base64
from io import BytesIO
```

questions.

- For questions of type `"image"`, uses `gpt-image-1` to generate an image and saves it under `static/generated/`. OpenAI Platform
- Displays everything on the page.
- Still supports “Download as PDF” (text-only Q&A).

python Copy code

```python
import os
import json
import base64
from io import BytesIO
from uuid import uuid4

from flask import Flask, render_template, request, send_file
from openai import OpenAI
import PyPDF2
from pdf2image import convert_from_bytes
import pytesseract

from reportlab.lib.pagesizes import letter
from reportlab.pdfgen import canvas

# ----------------------------
# Config
# ----------------------------
client = OpenAI()  # Uses OPENAI_API_KEY from env

app = Flask(__name__)

# Where we'll save generated images to serve in HTML
IMAGE_OUTPUT_DIR = os.path.join("static", "generated")
os.makedirs(IMAGE_OUTPUT_DIR, exist_ok=True)



# ----------------------------
# PDF extraction helpers
# ----------------------------

def pdf_to_text_standard(file_storage):
```

Copy code

```
app = Flask(__name__)

# Where we'll save generated images to serve in HTML
IMAGE_OUTPUT_DIR = os.path.join("static", "generated")
os.makedirs(IMAGE_OUTPUT_DIR, exist_ok=True)


# ----------------------------
# PDF extraction helpers
# ----------------------------

def pdf_to_text_standard(file_storage):
    """Extract text from a 'normal' PDF (typed discussion sheet)."""
    file_storage.stream.seek(0)
    reader = PyPDF2.PdfReader(file_storage.stream)
    text_chunks = []
    for page in reader.pages:
        page_text = page.extract_text() or ""
        text_chunks.append(page_text)
    return "\n".join(text_chunks)


def pdf_to_text_ocr(file_storage):
    """OCR extract text from a handwritten / scanned PDF (lecture notes
    file_storage.stream.seek(0)
    pdf_bytes = file_storage.read()

    images = convert_from_bytes(pdf_bytes)
    text_chunks = []
    for img in images:
        page_text = pytesseract.image_to_string(img)
        text_chunks.append(page_text)
    return "\n".join(text_chunks)


# ----------------------------
# OpenAI helpers
# ----------------------------

def generate_structured_questions(lecture_text: str, sheet_text: str) -
    """
```

Copy code

```
    for img in images:
        page_text = pytesseract.image_to_string(img)
        text_chunks.append(page_text)
    return "\n".join(text_chunks)


# ----------------------------
# OpenAI helpers
# ----------------------------

def generate_structured_questions(lecture_text: str, sheet_text: str) -
    """
    Ask the model to produce a mix of conceptual and image-based questi
    in STRICT JSON with this schema:

    {
      "questions": [
        {
          "type": "conceptual" | "image",
          "question": "<string>",
          "answer": "<string>",
          "image_prompt": "<string or null>"
        },
        ...
      ]
    }
    """

    max_chars = 15000
    lecture_text = lecture_text[:max_chars]
    sheet_text = sheet_text[:max_chars]

    system_msg = (
        "You are a university TA generating discussion questions. "
        "You ALWAYS output valid JSON following the schema I provide, "
        "with NO extra text before or after the JSON."
    )

    user_msg = f"""
You are helping design a new discussion sheet for an upper-division ma
```

Copy code

```
    """

    max_chars = 15000
    lecture_text = lecture_text[:max_chars]
    sheet_text = sheet_text[:max_chars]

    system_msg = (
        "You are a university TA generating discussion questions. "
        "You ALWAYS output valid JSON following the schema I provide, "
        "with NO extra text before or after the JSON."
    )

    user_msg = f"""
You are helping design a new discussion sheet for an upper-division ma

Inputs:
1. Lecture notes (OCR text; may be messy).
2. Existing discussion sheet containing questions and answers.

Goal:
- Identify key concepts from the lecture notes that are NOT already co
- Create a MIXTURE of:
  - Conceptual questions (no image needed).
  - Image-based questions that refer to a diagram/plot/table.

Requirements:
- Total of 3–4 questions.
- At least 1 question MUST be of type "image".
- For "image" questions:
  - Write a concise, concrete image description in the field "image_pr
    that can be given directly to an image-generation model.
  - The question itself should reference "the figure above" or similar
- For "conceptual" questions:
  - Set "image_prompt" to null.
- Do NOT duplicate or lightly rephrase existing discussion questions.
- Emphasize understanding/interpretation, not rote computation.

Return STRICT JSON with this schema:

{{
  "questions": [
```

Copy code

```
- For "image" questions:
  - Write a concise, concrete image description in the field "image_pr
    that can be given directly to an image-generation model.
  - The question itself should reference "the figure above" or similar
- For "conceptual" questions:
  - Set "image_prompt" to null.
- Do NOT duplicate or lightly rephrase existing discussion questions.
- Emphasize understanding/interpretation, not rote computation.

Return STRICT JSON with this schema:

{{
  "questions": [
    {{
      "type": "conceptual" or "image",
      "question": "string",
      "answer": "string",
      "image_prompt": "string or null"
    }},
    ...
  ]
}}

=== LECTURE NOTES (OCR TEXT) ===
\"\"\"{lecture_text}\"\"\"

=== EXISTING DISCUSSION SHEET (Q & A) ===
\"\"\"{sheet_text}\"\"\"
"""

    completion = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system_msg},
            {"role": "user", "content": user_msg},
        ],
        # If you want to try JSON mode explicitly and your model suppo
        # response_format={"type": "json_object"},
        max_completion_tokens=900,
        temperature=0.7,
    )
```

Copy code

```
    completion = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system_msg},
            {"role": "user", "content": user_msg},
        ],
        # If you want to try JSON mode explicitly and your model suppo
        # response_format={"type": "json_object"},
        max_completion_tokens=900,
        temperature=0.7,
    )

    content = completion.choices[0].message.content

    # Try to parse JSON. If it fails, you can add extra cleanup heuris
    data = json.loads(content)
    return data


def generate_images_for_questions(q_data: dict) -> dict:
    """
    For questions of type 'image', call the Images API with image_prom
    save the PNG to static/generated, and add 'image_url' to each ques
    """
    questions = q_data.get("questions", [])
    for q in questions:
        if q.get("type") != "image":
            continue

        prompt = q.get("image_prompt")
        if not prompt:
            continue

        img = client.images.generate(
            model="gpt-image-1",
            prompt=prompt,
            n=1,
            size="1024x1024",
        )
```

Copy code

```
            if q.get("type") != "image":
                continue

            prompt = q.get("image_prompt")
            if not prompt:
                continue

            img = client.images.generate(
                model="gpt-image-1",
                prompt=prompt,
                n=1,
                size="1024x1024",
            )

            image_bytes = base64.b64decode(img.data[0].b64_json)
            filename = f"qimage_{uuid4().hex}.png"
            path = os.path.join(IMAGE_OUTPUT_DIR, filename)
            with open(path, "wb") as f:
                f.write(image_bytes)

            # URL for Flask static serving
            q["image_url"] = f"/static/generated/{filename}"

    return q_data


def pretty_print_questions(q_data: dict) -> str:
    """
    Convert structured questions into a simple text block for displayi
    and for PDF download.
    """
    lines = []
    for i, q in enumerate(q_data.get("questions", []), start=1):
        lines.append(f"Q{i}. {q.get('question', '').strip()}")
        lines.append(f"A{i}. {q.get('answer', '').strip()}")
        lines.append("")  # blank line between
    return "\n".join(lines).strip()


# ----------------------------
# Flask routes
```

Copy code

```
    """
    Convert structured questions into a simple text block for displayi
    and for PDF download.
    """
    lines = []
    for i, q in enumerate(q_data.get("questions", []), start=1):
        lines.append(f"Q{i}. {q.get('question', '').strip()}")
        lines.append(f"A{i}. {q.get('answer', '').strip()}")
        lines.append("")  # blank line between
    return "\n".join(lines).strip()


# ----------------------------
# Flask routes
# ----------------------------

@app.route("/", methods=["GET", "POST"])
def index():
    questions_text = None
    questions_struct = None
    error = None

    if request.method == "POST":
        lecture_pdf = request.files.get("lecture_pdf")
        discussion_pdf = request.files.get("discussion_pdf")

        if not lecture_pdf or not discussion_pdf:
            error = "Please upload both the lecture notes PDF and the d
        else:
            try:
                # Handwritten lecture notes → OCR
                lecture_text = pdf_to_text_ocr(lecture_pdf)

                # Typed discussion sheet → normal extraction
                discussion_text = pdf_to_text_standard(discussion_pdf)

                if not lecture_text.strip():
                    error = "Could not extract text from the lecture no
                elif not discussion_text.strip():
                    error = "Could not extract text from the discussion
                else:
```

Copy code

```
                lecture_text = pdf_to_text_ocr(lecture_pdf)

                # Typed discussion sheet → normal extraction
                discussion_text = pdf_to_text_standard(discussion_pdf)

                if not lecture_text.strip():
                    error = "Could not extract text from the lecture n
                elif not discussion_text.strip():
                    error = "Could not extract text from the discussio
                else:
                    # 1) Ask ChatGPT for structured conceptual + image
                    q_struct = generate_structured_questions(lecture_t

                    # 2) Generate images for any image-based questions
                    q_struct = generate_images_for_questions(q_struct)

                    # 3) Human-readable block for <pre> and PDF export
                    questions_text = pretty_print_questions(q_struct)
                    questions_struct = q_struct

            except Exception as e:
                error = f"Something went wrong: {e}"

    # We also send questions_struct as JSON so the template can show i
    questions_json = json.dumps(questions_struct) if questions_struct

    return render_template(
        "index.html",
        questions_text=questions_text,
        questions_json=questions_json,
        error=error,
    )


@app.route("/download_pdf", methods=["POST"])
def download_pdf():
    """
    Take the generated questions text and turn it into a simple PDF.
    (For now this is text-only; images are shown in the web UI, but no
    embedded into the PDF yet.)
    """
```

Copy code

```
        questions_json=questions_json,
        error=error,
    )


@app.route("/download_pdf", methods=["POST"])
def download_pdf():
    """
    Take the generated questions text and turn it into a simple PDF.
    (For now this is text-only; images are shown in the web UI, but no
    embedded into the PDF yet.)
    """
    questions_text = request.form.get("questions_text", "")

    if not questions_text.strip():
        questions_text = "No questions were provided."

    buffer = BytesIO()
    c = canvas.Canvas(buffer, pagesize=letter)
    width, height = letter

    textobject = c.beginText()
    textobject.setTextOrigin(72, height - 72)  # 1-inch margin
    textobject.setLeading(14)

    for line in questions_text.split("\n"):
        if textobject.getY() <= 72:
            c.drawText(textobject)
            c.showPage()
            textobject = c.beginText()
            textobject.setTextOrigin(72, height - 72)
            textobject.setLeading(14)
        textobject.textLine(line)

    c.drawText(textobject)
    c.save()
    buffer.seek(0)

    return send_file(
        buffer,
        as_attachment=True,
```

```
            c.showPage()
            textobject = c.beginText()
            textobject.setTextOrigin(72, height - 72)
            textobject.setLeading(14)
        textobject.textLine(line)

    c.drawText(textobject)
    c.save()
    buffer.seek(0)

    return send_file(
        buffer,
        as_attachment=True,
        download_name="new_discussion_sheet.pdf",
        mimetype="application/pdf",
    )


if __name__ == "__main__":
    app.run(debug=True)
```

Copy code

---

3. Updated `templates/index.html` (show images + text + PDF download)

This version:

- Uses `questions_text` for the plain text block + PDF export.
- Uses `questions_json` (parsed on the server) to loop through questions and show:
  - Conceptual questions as plain text.
  - Image questions as **figure + question + answer**.

html

Copy code

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <title>Discussion Question Generator</title>
  <style>
    body {
```

- Uses `questions_json` (parsed on the server) to loop through questions and show:
  - Conceptual questions as plain text.
  - Image questions as **figure + question + answer**.

html Copy code

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <title>Discussion Question Generator</title>
  <style>
    body {
      font-family: system-ui, -apple-system, BlinkMacSystemFont, "Sego
      max-width: 900px;
      margin: 40px auto;
      padding: 0 16px;
      line-height: 1.5;
    }
    h1 {
      margin-bottom: 0.5rem;
    }
    .card {
      border: 1px solid #ddd;
      border-radius: 8px;
      padding: 16px;
      margin-top: 20px;
    }
    label {
      display: block;
      margin-top: 12px;
      font-weight: 500;
    }
    input[type="file"] {
      margin-top: 4px;
    }
    button {
      margin-top: 16px;
      padding: 8px 16px;
      border-radius: 6px;
      border: none;
```

Copy code

```
  display: block;
  margin-top: 12px;
  font-weight: 500;
}
input[type="file"] {
  margin-top: 4px;
}
button {
  margin-top: 16px;
  padding: 8px 16px;
  border-radius: 6px;
  border: none;
  cursor: pointer;
  font-size: 14px;
  font-weight: 600;
  background: #2563eb;
  color: white;
}
button:hover {
  background: #1d4ed8;
}
.secondary-button {
  background: #6b7280;
  margin-left: 8px;
}
.secondary-button:hover {
  background: #4b5563;
}
.error {
  color: #b91c1c;
  margin-top: 12px;
  font-weight: 500;
}
pre {
  white-space: pre-wrap;
  background: #f9fafb;
  padding: 12px;
  border-radius: 6px;
  border: 1px solid #e5e7eb;
}
.question-block {
```

Copy code

```
.error {
  color: #b91c1c;
  margin-top: 12px;
  font-weight: 500;
}
pre {
  white-space: pre-wrap;
  background: #f9fafb;
  padding: 12px;
  border-radius: 6px;
  border: 1px solid #e5e7eb;
}
.question-block {
  margin-top: 16px;
  padding-top: 12px;
  border-top: 1px dashed #e5e7eb;
}
.question-title {
  font-weight: 600;
  margin-bottom: 4px;
}
.question-type-tag {
  display: inline-block;
  font-size: 11px;
  padding: 2px 6px;
  border-radius: 4px;
  background: #e5e7eb;
  margin-left: 6px;
  vertical-align: middle;
}
.figure {
  margin-top: 8px;
  margin-bottom: 8px;
  text-align: center;
}
.figure img {
  max-width: 100%;
  height: auto;
  border-radius: 8px;
  border: 1px solid #e5e7eb;
}
```

Copy code

```
    }
    .figure img {
      max-width: 100%;
      height: auto;
      border-radius: 8px;
      border: 1px solid #e5e7eb;
    }
    .figure-caption {
      font-size: 12px;
      color: #6b7280;
      margin-top: 4px;
    }
  </style>
</head>
<body>
  <h1>Discussion Question Generator</h1>
  <p>
    Upload your <strong>lecture notes PDF</strong> (handwritten is fin
    and your <strong>existing discussion sheet PDF</strong>.
    The tool will generate a mix of:
  </p>
  <ul>
    <li>Conceptual questions</li>
    <li>Image-based questions with a generated diagram/figure</li>
  </ul>
  <p>
    You can also download the questions + answers as a PDF to use offl
  </p>

  <div class="card">
    <form method="post" enctype="multipart/form-data">
      <label>
        Lecture notes PDF (handwritten OK):
        <input type="file" name="lecture_pdf" accept="application/pdf"
      </label>

      <label>
        Existing discussion sheet PDF:
        <input type="file" name="discussion_pdf" accept="application/p
      </label>

      <button type="submit">Generate new questions</button>
```

Copy code

```
  <form method="post" enctype="multipart/form-data">
    <label>
      Lecture notes PDF (handwritten OK):
      <input type="file" name="lecture_pdf" accept="application/pdf"
    </label>

    <label>
      Existing discussion sheet PDF:
      <input type="file" name="discussion_pdf" accept="application/p
    </label>

    <button type="submit">Generate new questions</button>
  </form>

  {% if error %}
    <div class="error">{{ error }}</div>
  {% endif %}
</div>

{% if questions_text %}
  <div class="card">
    <h2>Generated Discussion Questions</h2>

    {# 1) Nicely rendered blocks with optional images #}
    {% if questions_json %}
      {% set q_data = questions_json | safe | loads %}
      {% for q in q_data.questions %}
        <div class="question-block">
          <div class="question-title">
            Q{{ loop.index }}.
            {{ q.question }}
            <span class="question-type-tag">
              {{ "Image-based" if q.type == "image" else "Conceptual'
            </span>
          </div>

          {% if q.type == "image" and q.image_url %}
            <div class="figure">
              <img src="{{ q.image_url }}" alt="Generated figure for
              <div class="figure-caption">
                Generated figure for this question.
```

Copy code

```
            {{ q.question }}
            <span class="question-type-tag">
              {{ "Image-based" if q.type == "image" else "Conceptual'
            </span>
          </div>

          {% if q.type == "image" and q.image_url %}
            <div class="figure">
              <img src="{{ q.image_url }}" alt="Generated figure for
              <div class="figure-caption">
                Generated figure for this question.
              </div>
            </div>
          {% endif %}

          <div style="margin-top: 4px;">
            <strong>Answer:</strong> {{ q.answer }}
          </div>
        </div>
      {% endfor %}
    {% endif %}

    <hr style="margin: 24px 0;" />

    {# 2) Plain text version (good for copying or checking) #}
    <h3>Plain text version</h3>
    <pre>{{ questions_text }}</pre>

    {# 3) Download as PDF (text only) #}
    <form method="post" action="/download_pdf">
      <textarea name="questions_text" style="display:none;">{{ quest
      <button type="submit" class="secondary-button">Download as PDF
    </form>
  </div>
{% endif %}
</body>
</html>
```

Note: The `loads` filter used here (`questions_json | safe | loads`) assumes you have Jinja's `|tojson` / JSON support available; if your environment doesn't, you can instead pre-parse on the server and pass `questions_struct` directly, or use

```
      <form method="post" action="/download_pdf">
        <textarea name="questions_text" style="display:none;">{{ quest
        <button type="submit" class="secondary-button">Download as PDF
      </form>
    </div>
  {% endif %}
</body>
</html>
```

Copy code

Note: The `loads` filter used here (`questions_json | safe | loads`) assumes you have Jinja's `|tojson` / JSON support available; if your environment doesn't, you can instead pre-parse on the server and pass `questions_struct` directly, or use `json.loads` inside the route and pass a Python dict instead of `questions_json` .

---

4. How this behaves now

1. You upload:
   - Handwritten lecture notes (GoodNotes / Notability export as PDF).
   - Existing discussion sheet PDF.
2. Backend:
   - OCRs lecture notes → text.
   - Extracts text from discussion PDF.
   - Calls ChatGPT to get **3–4 questions** in structured JSON:
     - Some `"type": "conceptual"` .
     - At least one `"type": "image"` with an `image_prompt` .
   - For each `"image"` question:
     - Calls OpenAI's **Images API** (`gpt-image-1`) with `image_prompt` and saves the PNG. OpenAI Platform
3. Website shows:
   - Each question with its answer.
   - For image-based questions, the **generated figure** above the question, similar vibe to "see the figure above, then answer…".
4. You can still download a **text-only PDF** of the questions/answers to bring to discussion.

   If you want, the next step can be: *embed those generated PNGs into the PDF layout as well.*

- At least one `"type": "image"` with an `image_prompt`.
- For each `"image"` question:
  - Calls OpenAI's **Images API** (`gpt-image-1`) with `image_prompt` and saves the PNG. OpenAI Platform

3. Website shows:
   - Each question with its answer.
   - For image-based questions, the **generated figure** above the question, similar vibe to "see the figure above, then answer...".
4. You can still download a **text-only PDF** of the questions/answers to bring to discussion.
   If you want, the next step can be: *embed those generated PNGs into the PDF layout as well.*

If you'd like, I can next:

- Add **images into the PDF** (so the offline sheet looks exactly like a full discussion handout), or
- Tune the prompt so image questions mimic your current course style (e.g., "draw the decision boundary", "annotate the diagram", "given this likelihood plot, answer...").

Sources

I would like you to add images into the pdf, and tune the prompt such that the image questions will miic the current course style.

It would be also good to have them latex because it is easier for me to read and if I want to distribute it to other students who want to use these resources, it would benefit them as well.

Fine tuning the prompt to make it fit more into something that I personally like.

Thought for 38s

Nice, this is a great upgrade:

I would like you to add images into the pdf, and tune the prompt such that the image questions will miic the current course style.

It would be also good to have them latex because it is easier for me to read and if I want to distribute it to other students who want to use these resources, it would benefit them as well.

Thought for 38s >

Nice, this is a great upgrade:

- **Images included in the PDF**
- **Questions & answers written in LaTeX style**
- **Image-based questions tuned to look like "real" upper-division ML/stat/CS discussion questions** (plots, decision boundaries, loss curves, etc.)

Below is a full version of the app updated to do all of that.

---

1. Updated dependencies

`requirements.txt`

txt Copy code

```
flask
openai>=1.0.0
PyPDF2
pdf2image
pytesseract
```

Install:

bash Copy code

```
pip install -r requirements.txt
```

System packages you need

```
PyPDF2
pdf2image
pytesseract
```

Copy code

Install:

bash Copy code

```
pip install -r requirements.txt
```

System packages you need

- **OCR** (as before):
  - macOS:

    bash Copy code

    ```
    brew install poppler tesseract
    ```

  - Ubuntu/Debian:

    bash Copy code

    ```
    sudo apt-get update
    sudo apt-get install poppler-utils tesseract-ocr
    ```

- **LaTeX (for `pdflatex`)**:
  - macOS: install **MacTeX**.
  - Ubuntu/Debian:

    bash Copy code

    ```
    sudo apt-get install texlive-latex-base texlive-latex-recommend
    ```

---

2. `app.py` – with LaTeX + embedded images in PDF

This version:

- Uses OCR for handwritten lecture notes.
- Extracts text from the existing discussion sheet.
- Generates **structured JSON** with both conceptual and image questions.
- Image questions are **tuned to look like standard ML/stat/CS discussion-style**

bash Copy code

```
sudo apt-get install texlive-latex-base texlive-latex-recommenc
```

---

2. `app.py` – with LaTeX + embedded images in PDF

This version:

- Uses OCR for handwritten lecture notes.
- Extracts text from the existing discussion sheet.
- Generates **structured JSON** with both conceptual and image questions.
- Image questions are **tuned to look like standard ML/stat/CS discussion-style figures** (loss curves, decision boundaries, scatterplots, etc.).
- Generates images via OpenAI's image API and saves them under `static/generated`.
- Builds a **LaTeX document** (article) with:
  - Sections for each question
  - `\includegraphics` for image questions
  - LaTeX-style Q/A text
- Runs `pdflatex` in a temp directory and returns the compiled PDF as a download.

python Copy code

```
import os
import json
import base64
import tempfile
import subprocess
import shutil
from io import BytesIO
from uuid import uuid4

from flask import Flask, render_template, request, send_file
from openai import OpenAI
import PyPDF2
from pdf2image import convert_from_bytes
import pytesseract
```

Copy code

```
import shutil
from io import BytesIO
from uuid import uuid4

from flask import Flask, render_template, request, send_file
from openai import OpenAI
import PyPDF2
from pdf2image import convert_from_bytes
import pytesseract

# ----------------------------
# Config
# ----------------------------
client = OpenAI()  # Uses OPENAI_API_KEY from env

app = Flask(__name__)

# Where we'll save generated images for web display
IMAGE_OUTPUT_DIR = os.path.join("static", "generated")
os.makedirs(IMAGE_OUTPUT_DIR, exist_ok=True)



# ----------------------------
# PDF extraction helpers
# ----------------------------

def pdf_to_text_standard(file_storage):
    """Extract text from a 'normal' PDF (typed discussion sheet)."""
    file_storage.stream.seek(0)
    reader = PyPDF2.PdfReader(file_storage.stream)
    text_chunks = []
    for page in reader.pages:
        page_text = page.extract_text() or ""
        text_chunks.append(page_text)
    return "\n".join(text_chunks)


def pdf_to_text_ocr(file_storage):
    """OCR extract text from a handwritten / scanned PDF (lecture notes
    file_storage.stream.seek(0)
    pdf_bytes = file_storage.read()
```

Copy code

```
    for page in reader.pages:
        page_text = page.extract_text() or ""
        text_chunks.append(page_text)
    return "\n".join(text_chunks)


def pdf_to_text_ocr(file_storage):
    """OCR extract text from a handwritten / scanned PDF (lecture notes
    file_storage.stream.seek(0)
    pdf_bytes = file_storage.read()

    images = convert_from_bytes(pdf_bytes)
    text_chunks = []
    for img in images:
        page_text = pytesseract.image_to_string(img)
        text_chunks.append(page_text)
    return "\n".join(text_chunks)


# ----------------------------
# OpenAI helpers
# ----------------------------

def generate_structured_questions(lecture_text: str, sheet_text: str) -
    """
    Ask the model to produce a mix of conceptual and image-based quest
    in STRICT JSON with schema:

    {
      "questions": [
        {
          "type": "conceptual" | "image",
          "question": "<LaTeX-ready string>",
          "answer": "<LaTeX-ready string>",
          "image_prompt": "<string or null>"
        },
        ...
      ]
    }
    """
```

Copy code

```
  "questions": [
    {
      "type": "conceptual" | "image",
      "question": "<LaTeX-ready string>",
      "answer": "<LaTeX-ready string>",
      "image_prompt": "<string or null>"
    },
    ...
  ]
}
"""

max_chars = 15000
lecture_text = lecture_text[:max_chars]
sheet_text = sheet_text[:max_chars]

system_msg = (
    "You are a university TA generating discussion questions for a
    "upper-division math/CS/stat / ML course. "
    "You ALWAYS output valid JSON following the schema the user pr
    "with NO extra text before or after the JSON."
)

user_msg = f"""
You are helping design a new discussion sheet for an upper-division ma

Inputs:
1. Lecture notes (OCR text; may be messy).
2. Existing discussion sheet containing questions and answers.

Goals:
- Identify key concepts from the lecture notes that are NOT already co
- Create a MIXTURE of:
  - Conceptual questions (no image needed).
  - Image-based questions that refer to a diagram/plot/table.

Total:
- 3–4 questions.
- At least 1 question MUST be of type "image".

Style requirements:
```

Copy code

```
- Create a MIXTURE of:
  - Conceptual questions (no image needed).
  - Image-based questions that refer to a diagram/plot/table.

Total:
- 3–4 questions.
- At least 1 question MUST be of type "image".

Style requirements:
- Mimic the style of typical discussion worksheets in advanced probabi
  Examples of good image-style questions:
  - A 2D scatterplot of data with a linear decision boundary; ask abou
  - A training/validation loss vs. epoch plot; ask about overfitting/e
  - A contour plot of a convex loss surface with gradient descent step
  - A likelihood surface over parameters; ask students to reason about
- Image prompts should be precise enough that an automatic figure will

LaTeX requirements:
- Write the question and answer text so it can be dropped directly int
  - Use $...$ for inline math, \\[ ... \\] for displayed math when hel
  - Use standard LaTeX notation (e.g., $\\mathbb{{R}}^d$, $\\theta$, $
  - Do NOT include a document preamble or \\begin{{document}} etc. — j

JSON schema:
Return STRICT JSON with this schema:

{{
  "questions": [
    {{
      "type": "conceptual" or "image",
      "question": "string (LaTeX-ready)",
      "answer": "string (LaTeX-ready)",
      "image_prompt": "string or null"
    }},
    ...
  ]
}}

Rules:
- Do NOT duplicate or lightly rephrase existing discussion questions.
- Emphasize understanding, interpretation, and reasoning, not rote com
```

Copy code

```
      "question": "string (LaTeX-ready)",
      "answer": "string (LaTeX-ready)",
      "image_prompt": "string or null"
    }},
    ...
  ]
}}

Rules:
- Do NOT duplicate or lightly rephrase existing discussion questions.
- Emphasize understanding, interpretation, and reasoning, not rote comp
- For "image" questions:
  - The "question" should explicitly reference "the figure above" or "
  - "image_prompt" should be a concise English description of the figu
    e.g.: "A 2D scatter plot of two Gaussian clusters in R^2, with a l
- For "conceptual" questions:
  - Set "image_prompt" to null.

=== LECTURE NOTES (OCR TEXT) ===
\"\"\"{lecture_text}\"\"\"

=== EXISTING DISCUSSION SHEET (Q & A) ===
\"\"\"{sheet_text}\"\"\"
"""

    completion = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system_msg},
            {
                "role": "user",
                "content": user_msg,
            },
        ],
        response_format={"type": "json_object"},
        max_completion_tokens=900,
        temperature=0.7,
    )

    data = json.loads(completion.choices[0].message.content)
    return data
```

Copy code

```
        ],
        response_format={"type": "json_object"},
        max_completion_tokens=900,
        temperature=0.7,
    )

    data = json.loads(completion.choices[0].message.content)
    return data


def generate_images_for_questions(q_data: dict) -> dict:
    """
    For questions of type 'image', call the Images API with image_prom
    save the PNG to static/generated, and add 'image_url' to each ques
    """
    questions = q_data.get("questions", [])
    for q in questions:
        if q.get("type") != "image":
            continue

        prompt = q.get("image_prompt")
        if not prompt:
            continue

        img = client.images.generate(
            model="gpt-image-1",
            prompt=prompt,
            n=1,
            size="1024x1024",
        )

        image_bytes = base64.b64decode(img.data[0].b64_json)
        filename = f"qimage_{uuid4().hex}.png"
        path = os.path.join(IMAGE_OUTPUT_DIR, filename)
        with open(path, "wb") as f:
            f.write(image_bytes)

        # URL for Flask static serving
        q["image_url"] = f"/static/generated/{filename}"

    return q_data
```

Copy code

```
            image_bytes = base64.b64decode(img.data[0].b64_jso..)
            filename = f"qimage_{uuid4().hex}.png"
            path = os.path.join(IMAGE_OUTPUT_DIR, filename)
            with open(path, "wb") as f:
                f.write(image_bytes)

            # URL for Flask static serving
            q["image_url"] = f"/static/generated/{filename}"

        return q_data


    def escape_latex(text: str) -> str:
        """
        Escape LaTeX special chars in plain text while leaving math delimi
        like $...$ alone as much as possible.
        Here we assume the model already uses $...$ for math; we mainly gu
        against stray &, %, #, _, {, }, ~, ^.
        """
        if text is None:
            return ""
        # Very simple; good enough for discussion sheets.
        replacements = {
            "&": r"\&",
            "%": r"\%",
            "#": r"\#",
            "_": r"\_",
            "{": r"\{",
            "}": r"\}",
            "~": r"\textasciitilde{}",
            "^": r"\textasciicircum{}",
        }
        out = text
        for k, v in replacements.items():
            out = out.replace(k, v)
        # Turn newlines into spaces to avoid weird paragraph breaks
        out = out.replace("\r", " ").replace("\n", " ")
        return out
```

Copy code

```
    }
    out = text
    for k, v in replacements.items():
        out = out.replace(k, v)
    # Turn newlines into spaces to avoid weird paragraph breaks
    out = out.replace("\r", " ").replace("\n", " ")
    return out


def build_latex_string(q_data: dict, image_map: dict) -> str:
    """
    Build a full LaTeX document (article) with questions + answers.
    image_map: {question_index -> local_image_filename}
    """
    header = r"""
\documentclass[11pt]{article}
\usepackage{amsmath, amssymb}
\usepackage{graphicx}
\usepackage[margin=1in]{geometry}
\usepackage{hyperref}

\title{Generated Discussion Sheet}
\author{}
\date{}

\begin{document}
\maketitle
"""

    lines = [header.strip(), ""]

    questions = q_data.get("questions", [])
    for i, q in enumerate(questions, start=1):
        q_text = q.get("question", "") or ""
        a_text = q.get("answer", "") or ""

        lines.append(f"\\section*{{Question {i}}}")

        # If we have an image for this question, include a figure
        if i in image_map:
            fig_name = image_map[i]
```

Copy code

```
    questions = q_data.get("questions", [])
    for i, q in enumerate(questions, start=1):
        q_text = q.get("question", "") or ""
        a_text = q.get("answer", "") or ""

        lines.append(f"\\section*{{Question {i}}}")

        # If we have an image for this question, include a figure
        if i in image_map:
            fig_name = image_map[i]
            lines.append(r"\begin{figure}[h]")
            lines.append(r"\centering")
            lines.append(f"\\includegraphics[width=0.8\\linewidth]{{{f
            lines.append(f"\\caption{{Generated figure for Question {i
            lines.append(r"\end{figure}")
            lines.append("")

        # Question text (already LaTeX-ish; lightly escaped)
        lines.append(r"\textbf{Prompt.} " + escape_latex(q_text))
        lines.append("")
        lines.append(r"\textbf{Solution.} " + escape_latex(a_text))
        lines.append("")
        lines.append("")

    lines.append(r"\end{document}")
    return "\n".join(lines)


def build_pdf_from_questions(q_data: dict) -> bytes:
    """
    Create a LaTeX doc with images embedded and compile it to PDF via
    Returns raw PDF bytes.
    """
    tmpdir = tempfile.mkdtemp()
    try:
        # Copy images into tmpdir and build mapping question_index ->
        image_map = {}
        for idx, q in enumerate(q_data.get("questions", []), start=1):
            if q.get("type") == "image" and q.get("image_url"):
                # image_url like /static/generated/foo.png
```

Copy code

```
            if q.get("type") == "image" and q.get("image_url"):
                # image_url like /static/generated/foo.png
                rel_path = q["image_url"].lstrip("/")
                src_path = os.path.join(app.root_path, rel_path)
                if os.path.exists(src_path):
                    dst_name = f"figure_q{idx}.png"
                    shutil.copy(src_path, os.path.join(tmpdir, dst_name
                    image_map[idx] = dst_name

        latex_source = build_latex_string(q_data, image_map)
        tex_path = os.path.join(tmpdir, "sheet.tex")
        with open(tex_path, "w", encoding="utf-8") as f:
            f.write(latex_source)

        # Run pdflatex
        cmd = ["pdflatex", "-interaction=nonstopmode", "sheet.tex"]
        proc = subprocess.run(
            cmd,
            cwd=tmpdir,
            stdout=subprocess.PIPE,
            stderr=subprocess.STDOUT,
            check=False,
        )

        pdf_path = os.path.join(tmpdir, "sheet.pdf")
        if not os.path.exists(pdf_path):
            # Compilation failed; raise an error with the log
            log = proc.stdout.decode("utf-8", errors="ignore")
            raise RuntimeError("pdflatex failed:\n" + log)

        with open(pdf_path, "rb") as f:
            pdf_bytes = f.read()

        return pdf_bytes

    finally:
        # Clean up temp directory
        shutil.rmtree(tmpdir, ignore_errors=True)


def pretty_print_questions(q_data: dict) -> str:
    """
```

Copy code

```
        with open(pdf_path, "rb") as f:
            pdf_bytes = f.read()

        return pdf_bytes

    finally:
        # Clean up temp directory
        shutil.rmtree(tmpdir, ignore_errors=True)


def pretty_print_questions(q_data: dict) -> str:
    """
    Convert structured questions into a simple text block for quick vi
    / copying in the web UI.
    """
    lines = []
    for i, q in enumerate(q_data.get("questions", []), start=1):
        lines.append(f"Q{i}. {q.get('question', '').strip()}")
        lines.append(f"A{i}. {q.get('answer', '').strip()}")
        lines.append("")
    return "\n".join(lines).strip()


# ----------------------------
# Flask routes
# ----------------------------

@app.route("/", methods=["GET", "POST"])
def index():
    questions_text = None
    questions_struct = None
    error = None

    if request.method == "POST":
        lecture_pdf = request.files.get("lecture_pdf")
        discussion_pdf = request.files.get("discussion_pdf")

        if not lecture_pdf or not discussion_pdf:
            error = "Please upload both the lecture notes PDF and the d
        else:
```

Copy code

```
questions_text = None
questions_struct = None
error = None

if request.method == "POST":
    lecture_pdf = request.files.get("lecture_pdf")
    discussion_pdf = request.files.get("discussion_pdf")

    if not lecture_pdf or not discussion_pdf:
        error = "Please upload both the lecture notes PDF and the d
    else:
        try:
            # Handwritten lecture notes → OCR
            lecture_text = pdf_to_text_ocr(lecture_pdf)

            # Typed discussion sheet → normal extraction
            discussion_text = pdf_to_text_standard(discussion_pdf)

            if not lecture_text.strip():
                error = "Could not extract text from the lecture no
            elif not discussion_text.strip():
                error = "Could not extract text from the discussion
            else:
                # 1) Ask ChatGPT for structured conceptual + image
                q_struct = generate_structured_questions(lecture_te

                # 2) Generate images for any image-based questions
                q_struct = generate_images_for_questions(q_struct)

                # 3) Human-readable block for <pre> and PDF export
                questions_text = pretty_print_questions(q_struct)
                questions_struct = q_struct

        except Exception as e:
            error = f"Something went wrong: {e}"

return render_template(
    "index.html",
    questions_text=questions_text,
    questions_struct=questions_struct,
    error=error,
```

Copy code

```
            except Exception as e:
                error = f"Something went wrong: {e}"

    return render_template(
        "index.html",
        questions_text=questions_text,
        questions_struct=questions_struct,
        error=error,
    )


@app.route("/download_pdf", methods=["POST"])
def download_pdf():
    """
    Take the structured questions (JSON) and build a LaTeX-based PDF w
    embedded images.
    """
    questions_json = request.form.get("questions_json", "")
    if not questions_json.strip():
        return "No question data provided.", 400

    try:
        q_data = json.loads(questions_json)
        pdf_bytes = build_pdf_from_questions(q_data)
    except Exception as e:
        # If anything goes wrong, return a simple error message
        return f"Error generating PDF: {e}", 500

    buffer = BytesIO(pdf_bytes)
    buffer.seek(0)

    return send_file(
        buffer,
        as_attachment=True,
        download_name="generated_discussion_sheet.pdf",
        mimetype="application/pdf",
    )


if __name__ == "__main__":
```

```
    buffer = BytesIO(pdf_bytes)
    buffer.seek(0)

    return send_file(
        buffer,
        as_attachment=True,
        download_name="generated_discussion_sheet.pdf",
        mimetype="application/pdf",
    )


if __name__ == "__main__":
    # Make sure OPENAI_API_KEY is set in your environment
    # e.g. export OPENAI_API_KEY="sk-..."
    app.run(debug=True)
```

Copy code

---

3. `templates/index.html` – now passes full JSON to PDF route

This template:

- Shows questions + answers.
- Shows images inline for image questions.
- Sends the **full structured JSON** as a hidden field to `/download_pdf`, which then builds the LaTeX+PDF.

html

Copy code

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <title>Discussion Question Generator</title>
  <style>
    body {
      font-family: system-ui, -apple-system, BlinkMacSystemFont, "Sego
      max-width: 900px;
      margin: 40px auto;
      padding: 0 16px;
      line-height: 1.5;
    }
```

```
<style>
  body {
    font-family: system-ui, -apple-system, BlinkMacSystemFont, "Sego
    max-width: 900px;
    margin: 40px auto;
    padding: 0 16px;
    line-height: 1.5;
  }
  h1 {
    margin-bottom: 0.5rem;
  }
  .card {
    border: 1px solid #ddd;
    border-radius: 8px;
    padding: 16px;
    margin-top: 20px;
  }
  label {
    display: block;
    margin-top: 12px;
    font-weight: 500;
  }
  input[type="file"] {
    margin-top: 4px;
  }
  button {
    margin-top: 16px;
    padding: 8px 16px;
    border-radius: 6px;
    border: none;
    cursor: pointer;
    font-size: 14px;
    font-weight: 600;
    background: #2563eb;
    color: white;
  }
  button:hover {
    background: #1d4ed8;
  }
  .secondary-button {
    background: #6b7280;
```

Copy code

```
}
button:hover {
  background: #1d4ed8;
}
.secondary-button {
  background: #6b7280;
  margin-left: 8px;
}
.secondary-button:hover {
  background: #4b5563;
}
.error {
  color: #b91c1c;
  margin-top: 12px;
  font-weight: 500;
}
pre {
  white-space: pre-wrap;
  background: #f9fafb;
  padding: 12px;
  border-radius: 6px;
  border: 1px solid #e5e7eb;
}
.question-block {
  margin-top: 16px;
  padding-top: 12px;
  border-top: 1px dashed #e5e7eb;
}
.question-title {
  font-weight: 600;
  margin-bottom: 4px;
}
.question-type-tag {
  display: inline-block;
  font-size: 11px;
  padding: 2px 6px;
  border-radius: 4px;
  background: #e5e7eb;
  margin-left: 6px;
  vertical-align: middle;
}
```

Copy code

```
      font-size: 11px;
      padding: 2px 6px;
      border-radius: 4px;
      background: #e5e7eb;
      margin-left: 6px;
      vertical-align: middle;
    }
    .figure {
      margin-top: 8px;
      margin-bottom: 8px;
      text-align: center;
    }
    .figure img {
      max-width: 100%;
      height: auto;
      border-radius: 8px;
      border: 1px solid #e5e7eb;
    }
    .figure-caption {
      font-size: 12px;
      color: #6b7280;
      margin-top: 4px;
    }
  </style>
</head>
<body>
  <h1>Discussion Question Generator</h1>
  <p>
    Upload your <strong>lecture notes PDF</strong> (handwritten is fin
    and your <strong>existing discussion sheet PDF</strong>.
    The tool will generate a mix of:
  </p>
  <ul>
    <li>Conceptual questions (LaTeX-style)</li>
    <li>Image-based questions with a generated figure (decision bounda
  </ul>
  <p>
    You can download everything as a LaTeX-generated PDF with figures
    ready to share with students.
  </p>
```

Copy code

```
  <li>Image-based questions with a generated figure (dec
</ul>
<p>
  You can download everything as a LaTeX-generated PDF with figures
  ready to share with students.
</p>

<div class="card">
  <form method="post" enctype="multipart/form-data">
    <label>
      Lecture notes PDF (handwritten OK):
      <input type="file" name="lecture_pdf" accept="application/pdf"
    </label>

    <label>
      Existing discussion sheet PDF:
      <input type="file" name="discussion_pdf" accept="application/p
    </label>

    <button type="submit">Generate new questions</button>
  </form>

  {% if error %}
    <div class="error">{{ error }}</div>
  {% endif %}
</div>

{% if questions_text and questions_struct %}
  <div class="card">
    <h2>Generated Discussion Questions</h2>

    {# 1) Nicely rendered blocks with optional images #}
    {% for q in questions_struct.questions %}
      <div class="question-block">
        <div class="question-title">
          Q{{ loop.index }}.
          {{ q.question }}
          <span class="question-type-tag">
            {{ "Image-based" if q.type == "image" else "Conceptual"
          </span>
        </div>
```

Copy code

```
            {{ q.question }}
            <span class="question-type-tag">
              {{ "Image-based" if q.type == "image" else "Conceptual" }
            </span>
          </div>

          {% if q.type == "image" and q.image_url %}
            <div class="figure">
              <img src="{{ q.image_url }}" alt="Generated figure for qu
              <div class="figure-caption">
                Generated figure for this question.
              </div>
            </div>
          {% endif %}

          <div style="margin-top: 4px;">
            <strong>Answer:</strong> {{ q.answer }}
          </div>
        </div>
      {% endfor %}

      <hr style="margin: 24px 0;" />

      {# 2) Plain text version (good for quick copying) #}
      <h3>Plain text version (LaTeX-style)</h3>
      <pre>{{ questions_text }}</pre>

      {# 3) Download as LaTeX-based PDF (with figures embedded) #}
      <form method="post" action="/download_pdf">
        <textarea name="questions_json" style="display:none;">{{ quest
        <button type="submit" class="secondary-button">Download PDF (La
      </form>
    </div>
  {% endif %}
</body>
</html>
```

---

4. How to run & use

```
        <button type="submit" class="secondary-button">Download PDF (La
      </form>
    </div>
  {% endif %}
</body>
</html>
```

Copy code

---

4. How to run & use

1. Set your API key:

   bash Copy code

   ```
   export OPENAI_API_KEY="sk-..."
   ```

2. Run the app:

   bash Copy code

   ```
   python app.py
   ```

3. Open in browser:
   `http://127.0.0.1:5000/`
4. Upload:
   - Handwritten lecture notes PDF (GoodNotes / Notability export).
   - Existing discussion sheet PDF.
5. Click **“Generate new questions”**:
   - The page shows conceptual + image-based questions.
   - Image questions have generated figures inline.
6. Click **“Download PDF (LaTeX)”**:
   - Flask builds a LaTeX file, embeds the figures, runs `pdflatex`, and you get a **nice, LaTeX-formatted PDF** you can hand out or upload.

If you want, next step could be:

- Add a **course header** (e.g., “CS 182 — Week 7 Discussion”) with date.
- Add an option to **download the raw `.tex` file** alongside the PDF so you can tweak formatting by hand.